C++

FILE HANDLING

A Complete Guide to File I/O, Streams, Binary Data & Real-World Applications

Covering: Streams • fstream • Text/Binary Files • Error Handling • Real-Life Projects

Comprehensive C++ Programming Series

TABLE OF CONTENTS

SECTION 01

Introduction to File Handling

Why files matter • Persistence • Real-world motivation

WHAT IS FILE HANDLING?

File Handling in C++ refers to the set of operations that allow programs to **create, read, write, append, and delete** data in files stored on permanent storage (disk). Unlike variables which lose data on program exit, files provide **persistent storage.**

Persistence

Data survives program termination. A student record saved today can be retrieved tomorrow.

Data Exchange

Files enable communication between programs, systems, and organisations via CSV, JSON, XML formats.

Large Datasets

RAM is limited. Files store gigabytes of data that cannot fit in memory simultaneously.

WHY FILE HANDLING? — REAL-WORLD SCENARIOS

University System

Store student records, grades, attendance permanently across semesters.

Banking Application

Transaction logs, account balances, audit trails written to secure files.

Game Save System

Save player progress, inventory, level data so users resume where they left off.

Data Analysis

Read CSV datasets with millions of rows, process them in batches.

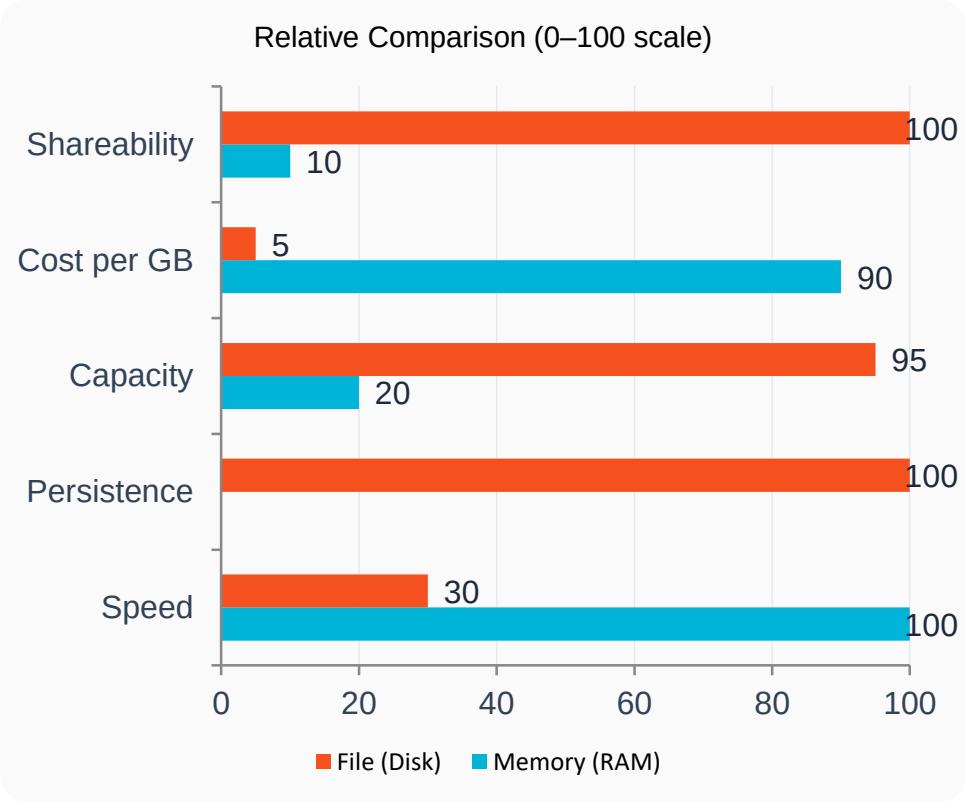
Configuration Files

Programs read settings from .ini/.json files at startup without recompiling.

Log Files

Web servers, operating systems record events, errors, and access in log files.

MEMORY vs FILE STORAGE — KEY DIFFERENCES



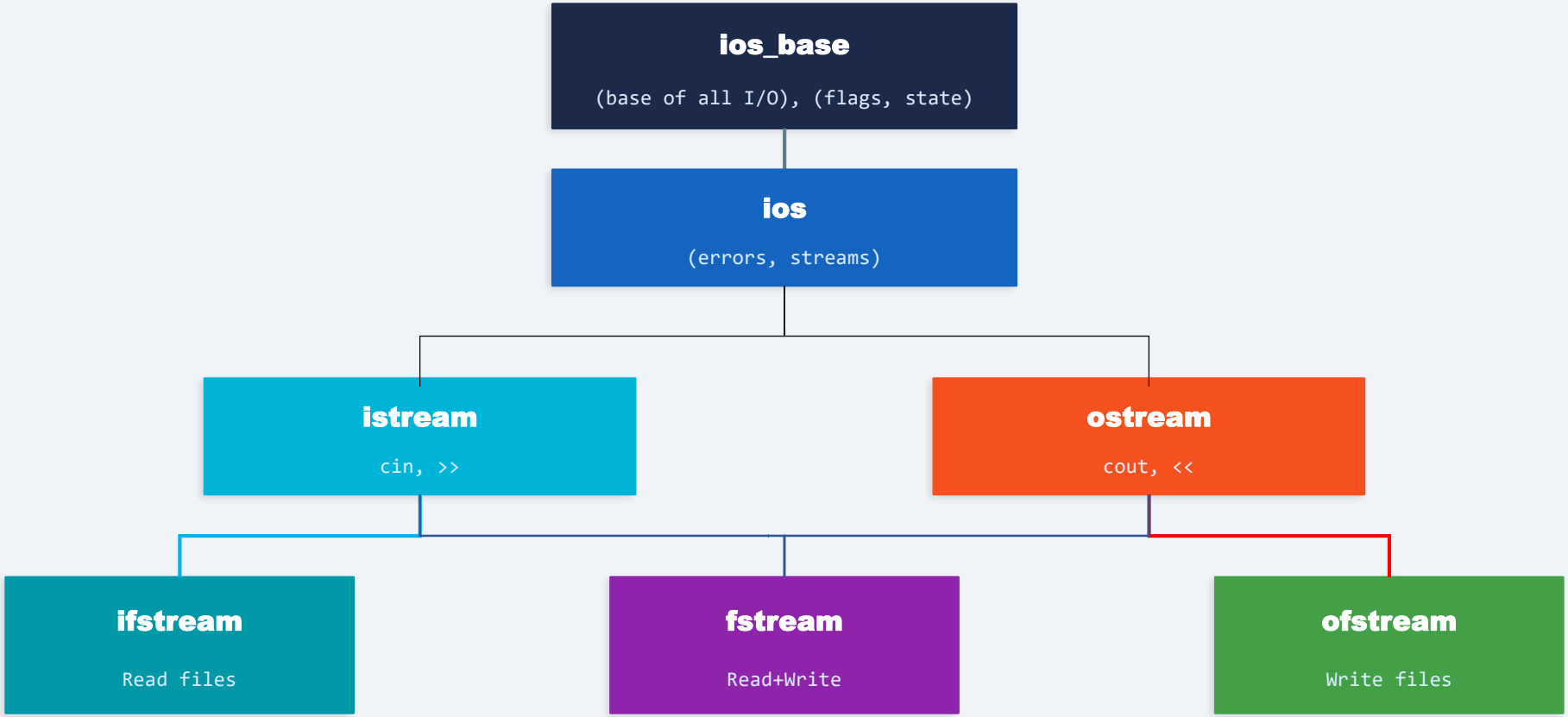
Feature	RAM (Memory)	File (Disk)
Speed	Nanoseconds	Milliseconds
Persistence	Lost on exit ✗	Permanent ✓
Capacity	GBs (limited)	TBs (vast)
Access	Random, fast	Sequential/ Random
Cost	Expensive	Very cheap
Sharing	Process-local	Cross-program/ network

SECTION 02

C++ Stream Classes

fstream • ifstream • ofstream • The stream hierarchy

C++ STREAM CLASS HIERARCHY



<fstream> header required for file classes

THE THREE FILE STREAM CLASSES

ifstream

Input File Stream

Used to READ data FROM a file. Inherits from istream. Supports >> operator and getline().

Key Methods:

- open(path/filename, flag/mode)
- close()
- >> (extraction)
- getline()
- read()
- get()
- eof(), fail()
- is_open()

ofstream

Output File Stream

Used to WRITE data TO a file. Inherits from ostream. Supports << operator for output.

Key Methods:

- open(path/filename, flag/mode)
- close()
- << (insertion)
- write()
- put()
- flush()
- tellp(), seekp()
- is_open()

fstream

File Stream (Both)

Used for BOTH reading AND writing. Inherits from iostream. Most flexible — requires explicit mode flags.

Key Methods:

- open(filename, mode)
- close()
- >> and <<
- read() + write()
- seekg() + seekp()
- tellg() + tellp()
- All error flags
- is_open()
- getline()

SECTION 03

Opening & Closing Files

open() • File modes • ios flags • Checking success

FILE OPENING MODES (ios FLAGS)

ios::in

→ **Open for reading**

Effect: File must exist

ios::out

→ **Open for writing**

Effect: Creates/truncates (overwrite) file

ios::app

→ **Append — write at end**

Effect: Creates if not exists; never truncates

ios::ate

→ **At-end — seek to end on open**

Effect: Can still write anywhere

ios::trunc

→ **Truncate — erase existing content before writing new**

Effect: Used with ios::out by default

ios::binary

→ **Binary mode — no newline translation**

Effect: Essential for binary file I/O

Modes can be combined with | operator to specify multiple flags at once: `fstream f("x.dat", ios::in | ios::out | ios::binary);`

OPENING & CLOSING FILES — SYNTAX & BEST PRACTICES

OPENING FILES — MULTIPLE WAYS

```
// Method 1: Constructor
ifstream fin("students.txt");
ofstream fout("results.txt");
fstream  rw("data.bin",
ios::in|ios::out|ios::binary);

// Method 2: open() function
ifstream fin2;
fin2.open("log.txt", ios::in);

// ── Always check if file opened ──
if (!fin.is_open()) {
 cerr << "Error: Cannot open file!\n";
 return 1;    // exit gracefully
}
```

```
// ──── Reading all lines ────
string line;
while (getline(fin, line)) {
      cout << line << "\n";
}

// ──── Writing data ────
fout << "Name: Ali\n";
fout << "Score: " << 95 << "\n";

// ──── Always close! ────
fin.close();
fout.close();

// ──── Or use RAII scope ────
{   // Block scope
    ifstream scoped("temp.txt");
  //... file closed automatically at }
}
```

BEST PRACTICES

- ✓ Always check is_open()
- ✓ Use RAII / scope blocks
- ✓ close() when done
- ✓ Prefer relative paths
- ⚠ ios::out truncates by default
- ⚠ Binary ≠ Text mode
- ✗ **Never ignore open failures**
- ✗ **Don't hard-code absolute paths**

SECTION 04

Text File I/O

Reading lines • Parsing tokens • Writing formatted data

TEXT FILE READING — FOUR TECHNIQUES

1. Extraction Operator >>

```
ifstream fin("data.txt");
int n; string s;
while(fin >> n >> s){
  cout << n << " " << s;
}
// Skips whitespace, newlines
```

2. getline() — Whole Lines

```
ifstream fin("data.txt");
string line;
while(getline(fin,line)){
  cout << line << "\n";
}
// Preserves spaces in line
```

3. get() — Character by Character

```
ifstream fin("data.txt");
char ch;
while(fin.get(ch)){
  if(ch=='\n') cout<<"[EOF]";
  else cout << ch;
}
// Reads every byte including \n
```

4. read() block — Fixed Bytes

```
ifstream fin("data.txt");
char buf[256];
fin.read(buf, 255);
buf[fin.gcount()] = '\0';
cout << buf;
// gcount() = bytes read
```

Choose based on: >> for tokens, getline() for lines, get() for exact chars, read() for large binary blocks

TEXT FILE WRITING — FORMATTED OUTPUT

WRITING FORMATTED TEXT DATA

```cpp
#include <iostream>
#include <fstream>
#include <iomanip>    // for setw, fixed
using namespace std;

int main() {
    ofstream fout("report.txt");

    if (!fout.is_open()) {
        cerr << "Cannot open file!\n";
        return 1;
    }

    // ── Plain text ──────────────────
    fout << "=== Sales Report ===" << "\n";
    fout << "Date: 23.04.26\n\n";

    // ── Formatted table ─────────────
    fout << left << setw(15) << "Product"
         << right<< setw(8)  << "Price"
         << right<< setw(6)  << "Qty"
         << "\n";
    fout << string(30, '-') << "\n";

    fout << left << setw(15) << "Laptop"
         << right<< setw(8)  << fixed
         << setprecision(2)  << 75000.00
         << right<< setw(6)  << 5
         << "\n";

    // ── Append mode example ─────────
    // ofstream fapp("log.txt", ios::app);
//fapp << "[23.04.26 10:30] User login\n";

    fout.close();
    cout << "Report written!\n";
    return 0;
}
```

OUTPUT: report.txt

```
=== Sales Report ===
Date: 23.04.26

Product         Price   Qty
------------------------------
Laptop       75000.00     5
```

iomanip Cheat Sheet

- setw(n) — field width
- left/right — alignment
- fixed — decimal point
- setprecision(n) — decimals
- setfill('x') — fill char

SECTION 04

CSV File I/O

Reading • Writing

Comma Separate Values (CSV)

- CSV stands for **Comma-Separated Values**, and it is a file format used to store data in a tabular form
- In a CSV file, each line represents a row in the table, and each field within a line is separated by a comma or other delimiter character

Why CSV

- CSV files provide an easy way to exchange data between different software programs, since most applications support importing and exporting data in this format
- CSV files are simple & lightweight, making them easy to work with and manipulate using a variety of tools and programming languages
- CSV files are plain text files, they can be easily read and edited using any text editor, which makes them a flexible and accessible way to store and organize data

Writing CSV

- Writing data to CSV is as easy as writing in normal file, except it separates each field with comma (,) e.g. consider an `ofstream` object `fileWriter`:

```
fileWrite << “Shehzad, Ahmad, 30, Lecturer, CS” << endl;
fileWrite << “Zaid, Yousaf, 25, Lecturer, EE” << endl;
fileWrite << “Muhammad, Ishaq, 40, Lecturer, CSE” << endl;
fileWrite << “Usman, Ali, 35, Lecturer, TE” << endl;
```

- This code will store this data as comma separated values in the file

Reading CSV

- Reading data from CSV is also as easy as reading in normal file, except we keep care of the commas
- Consider an `ifstream` object `fileRead`:

```
string name, age, desig;
while(getline(fileRead,name,',') && getline(fileRead,age,',') &&
      getline(fileRead,desig))
{
        cout<<"Name: "<<name<<endl;
        cout<<"Age: "<<age<<endl;
        cout<<"Designation: "<<desig<<endl;
        cout<<"---------------"<<endl;
}
```

- This code will read each record from CSV as line and will separate each field and assign it to specific variable.

REAL PROJECT: Student Record System (Text--CSV Files)

STUDENT RECORD SYSTEM — TEXT FILE I/O

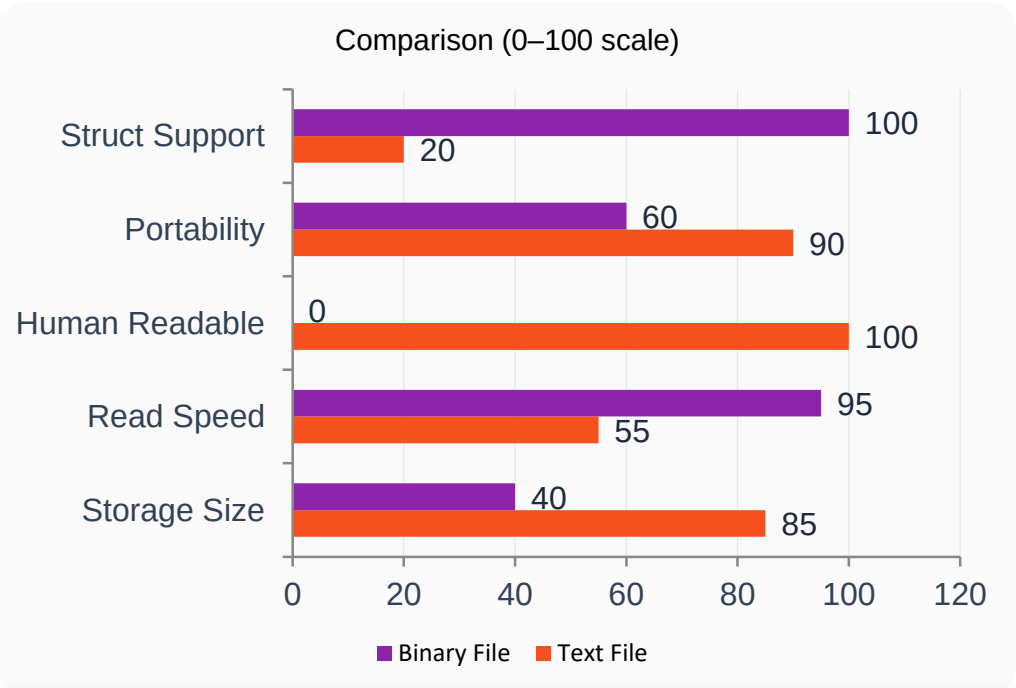
```cpp
#include <iostream>
#include <fstream>
#include <sstream>
using namespace std;

struct Student { string name; int id, age; float gpa; };

// Save all students to file
void saveStudents(const Student arr[], int n, const string&
filename) {
    ofstream fout(filename);
    for (int i = 0; i < n; i++)
        fout << arr[i].id << "," << arr[i].name << ","
             << arr[i].age << "," << arr[i].gpa << "\n";
    cout << n << " students saved to " << filename << "\n";
}

// Load students from file
int loadStudents(Student arr[], const string& filename) {
    ifstream fin(filename);
    if (!fin.is_open()) { cerr << "File not found!\n"; return 0; }
    int n = 0; string line;
    while (getline(fin, line) && n < 100) {
        stringstream ss(line);
        string tok;
        getline(ss, tok, ','); arr[n].id   = stoi(tok);
        getline(ss, tok, ','); arr[n].name = tok;
        getline(ss, tok, ','); arr[n].age  = stoi(tok);
        getline(ss, tok, ','); arr[n].gpa  = stof(tok);
        n++;
    }
    cout << n << " students loaded.\n";
    return n;
}

int main() {
    Student students[] = {{"Ali",1,20,3.8}, {"Sara",2,21,3.9},
                          {"Zain",3,22,3.5}};
    saveStudents(students, 3, "students.csv");
    Student loaded[100];
    int count = loadStudents(loaded, "students.csv");
    for (int i=0;i<count;i++)
        cout << loaded[i].id <<" "<< loaded[i].name <<" GPA:"<<
loaded[i].gpa <<"\n";
}
```

SECTION 05

Binary File I/O

read() • write() • Struct serialization • Random access

BINARY vs TEXT FILES



Aspect	Text File	Binary File
Content	Human-readable ASCII	Raw bytes / bit patterns
int value 65	Stored as '6','5' (2B)	Stored as 0x41 (1B)
Newlines.	\n translated by OS	No translation
Open mode	Default	ios::binary required
Use case	Config, logs, CSV	Images, structs, DB
Editing	Any text editor	Hex editor needed

TEXT vs BINARY — QUICK EXAMPLE

```
// Text:   ofstream f("x.txt");       f << 65;     // writes "65" (2 bytes)
// Binary: ofstream f("x.bin",ios::binary);
           f.write((char*) &n, sizeof(n));          // writes 4 bytes
```

BINARY FILE I/O — read() AND write() [×: >> or <<]

BINARY FILE — STRUCT SERIALIZATION

```
#include <iostream>
#include <fstream>
using namespace std;

struct Employee {
    char   name[50];
    int    id;
    float  salary;
    int    department;
};  // sizeof(Employee) = 50+4+4+4 = 62 bytes (with possible
padding)

// ── Write binary
void writeEmployee(const Employee& emp, const string& filename)
{
 ofstream fout(filename, ios::binary | ios::app); //append mode
    if (!fout) { cerr << "Write error!\n"; return; }
    fout.write(reinterpret_cast<const char*>(&emp),
               sizeof(Employee));
    cout << "Employee " << emp.name << " saved.\n";
}
```

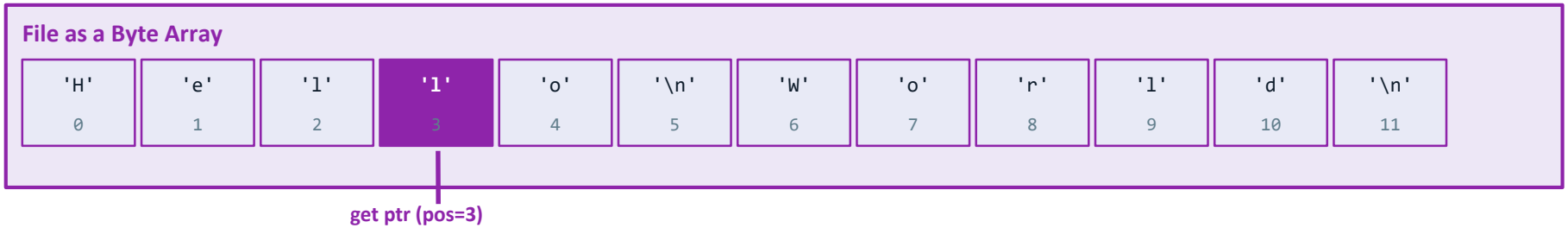
```
    // ── Read all employees
void readAllEmployees(const string& filename) {
    ifstream fin(filename, ios::binary);
    if (!fin) { cerr << "Read error!\n"; return; }

    Employee emp;
    int count = 0;
    while (fin.read(reinterpret_cast<char*>(&emp),
           sizeof(Employee))) {
        cout << "ID:"    << emp.id
             << " Name:" << emp.name
             << " Sal:"  << emp.salary
             << " Dept:" << emp.department << "\n";
        count++;
    }
    cout << count << " records read.\n";
}
int main() {
    Employee e1 = {"Ali Khan",  101, 75000.0f, 2};
    Employee e2 = {"Sara Baig", 102, 82000.0f, 1};
    Employee e3 = {"Zain Mir",  103, 91000.0f, 3};
    writeEmployee(e1, "employees.bin");
    writeEmployee(e2, "employees.bin");
    writeEmployee(e3, "employees.bin");
    cout << "\n--- All Employees ---\n";
    readAllEmployees("employees.bin");
}
```

SECTION 06

File Pointers & Seeking

seekg • seekp • tellg • tellp • Random Access

FILE POINTERS — seekg, seekp, tellg, tellp



seekg(pos)

Move GET pointer to absolute byte position

```
fin.seekg(0);            // go to start
fin.seekg(0, ios::end);  // go to end
fin.seekg(-4, ios::cur); // 4 bytes back
```

seekg(off, dir)

dir: ios::beg, ios::cur, ios::end

```
fin.seekg(10, ios::beg); // Seek from beginning
fin.seekg(5, ios::cur);  // Seek from current
```

seekp(pos)

Move PUT pointer (for writing)

```
fout.seekp(0);           // write from start
fout.seekp(0,ios::end);  // append to end
```

tellg() / tellp()

Return current pointer position as streampos

```
streampos pos = fin.tellg();
// save position
// ... read some data ...
fin.seekg(pos); // restore
```

RANDOM ACCESS — Read/Update Any Record Directly

RANDOM ACCESS — O(1) RECORD LOOKUP

```
#include <iostream>
#include <fstream>
using namespace std;

struct Product { int id; char name[40]; float price; int stock;};
const string FILE = "inventory.bin";

// Seek directly to record n (zero-indexed)
void readRecord(int n) {
    ifstream fin(FILE, ios::binary);
    fin.seekg(n * sizeof(Product)); //jump to exact byte position
    Product p;
    fin.read(reinterpret_cast<char*>(&p), sizeof(Product));
    cout << "Record " << n << ": " << p.name << " | Price: " <<
            p.price << " | Stock: " << p.stock << "\n";
}

// Update only the stock of record n — O(1) direct access!
void updateStock(int n, int newStock) {
    fstream file(FILE, ios::in | ios::out | ios::binary);
    streampos pos = n * sizeof(Product);
    file.seekg(pos);                    // seek for reading
    Product p;
    file.read(reinterpret_cast<char*>(&p), sizeof(Product));
    p.stock = newStock;                 // modify
    file.seekp(pos);                    // seek back for writing
file.write(reinterpret_cast<const char*>(&p), sizeof(Product));
    cout << "Stock updated for record " << n << "\n";
}

// Calculate total records in file
int totalRecords() {
    ifstream fin(FILE, ios::binary | ios::ate);  // open at end
    return fin.tellg() / sizeof(Product);   // size / record size
}

int main() {
    // Write initial data
ofstream fout(FILE, ios::binary);
Product products[]={{1,"Laptop",75000,50},
                    {2,"Phone",35000,120},{3,"Tablet",25000,75}};
 for (auto& p : products)
  fout.write(reinterpret_cast<const char*>(&p), sizeof(Product));
    fout.close();

    cout << "Total records: " << totalRecords() << "\n";
    readRecord(1);              // Read Phone directly — O(1)!
    updateStock(1, 115);        // Update Phone stock
    readRecord(1);              // Verify update
}
```

SECTION 07

Error Handling in File I/O

State flags • Exception mode • Robust programs

STREAM STATE FLAGS & ERROR HANDLING

good()

All bits clear — stream is fully functional

When: After successful operation

eof()

End-Of-File reached — no more data to read

When: After reading past last byte

fail()

Logical error — wrong type, format mismatch

When: int read where string present

bad()

Fatal I/O error — hardware failure, permissions

When: Disk full, hardware error

clear()

Reset all error flags to good state

When: After handling an error

rdstate()

Returns bitmask of all current state bits

When: For detailed state inspection

```
State hierarchy: good() → fail() → bad() — once bad, clear() required.
```

ROBUST FILE ERROR HANDLING — CODE PATTERNS

FOUR ERROR-HANDLING PATTERNS

```
#include <iostream>
#include <fstream>
#include <stdexcept>
using namespace std;

// ── Pattern 1: Boolean check (most common) ──────────────────
void pattern1() {
    ifstream fin("data.txt");
    if (!fin) {                          // operator bool() checks !fail()
        cerr << "ERROR: Cannot open data.txt\n";
        return;
    }
    int n;
    while (fin >> n) {                   // loop exits on eof or fail
        cout << n << "\n";
    }
    if (fin.bad())  cerr << "Fatal I/O error!\n";
    if (fin.fail() && !fin.eof()) cerr << "Format error (not eof)!\n";
}

// ── Pattern 2: Exception mode ───────────────────────────────
void pattern2() {
    ifstream fin;
   fin.exceptions(ifstream::failbit | ifstream::badbit); //throw on errors
    try {
        fin.open("data.txt");
        string line;
        while (getline(fin, line)) cout << line << "\n";
    }
catch (const ifstream::failure& e) {
        cerr << "File I/O exception: " << e.what() << "\n";
    }
}

// ── Pattern 3: Retry with clear() ───────────────────────────
void pattern3() {
    fstream f("mixed.txt");
    int n; string s;
    f >> n;                                  // try to read int
    if (f.fail()) {
        f.clear();                           // clear fail bit
        f.seekg(0);                          // go back to start
        f >> s;                              // try as string
        cout << "Read string instead: " << s << "\n";
    }
}

// ── Pattern 4: RAII wrapper ─────────────────────────────────
class SafeFile {
    ifstream fin;
public:
    SafeFile(const string& name) : fin(name) {
        if (!fin) throw runtime_error("Cannot open: " + name);
    }
    bool readLine(string& line) { return (bool)getline(fin, line); }
    ~SafeFile() { /* fin closes automatically */ }
};
```

SECTION 08

Real-Life Projects

Log System • CSV Parser • Config Files • Student DB

PROJECT 1: Application Logger (Like Web Server Logs)

PROJECT 1: Application Logger

```cpp
#include <iostream>
#include <fstream>
#include <ctime>
#include <string>
using namespace std;

enum LogLevel { INFO, WARNING, ERROR_LOG, CRITICAL };

class Logger {
    string filename;
    string levelStr(LogLevel lv) {
        switch(lv){case INFO:return"[INFO]";case WARNING:return"[WARN]";
                   case ERROR_LOG:return"[ERR ]";default:return"[CRIT]";}
    }
    string timestamp() {
        time_t now = time(nullptr);
        char buf[20];
        strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localtime(&now));
        return string(buf);
    }
public:
    Logger(const string& file) : filename(file) {}
    void log(LogLevel lv, const string& message) {
        ofstream fout(filename, ios::app);        // Always append
        if (!fout) { cerr << "Logger failed!\n"; return; }
        fout << timestamp() << " " << levelStr(lv) << " " <<message << "\n";
        fout.flush();                    // Ensure write to disk
        // Also echo to console for ERROR and above
      if (lv >= ERROR_LOG)
        cerr << timestamp() <<" " <<levelStr(lv) << " " <<message << "\n";
    }
};

int main() {
    Logger logger("app.log");
    logger.log(INFO,     "Server started on port 8080");
    logger.log(INFO,     "User 'ali' logged in from 192.168.1.5");
    logger.log(WARNING,  "Disk usage at 85%");
    logger.log(ERROR_LOG,"Database connection timeout after 30s");
    logger.log(CRITICAL, "Memory allocation failed — restarting
service");
    cout << "Logs written to app.log\n";
}



/* ========================================================
   app.log output:
   2024-01-15 10:30:01 [INFO] Server started on port 8080
   2024-01-15 10:30:15 [INFO] User 'ali' logged in from 192.168.1.5
   2024-01-15 10:31:02 [WARN] Disk usage at 85%
   2024-01-15 10:32:44 [ERR ] Database connection timeout after 30s
   2024-01-15 10:33:01 [CRIT] Memory allocation failed — restarting
service */
```

PROJECT 2: Binary Student Database — CRUD Operations

PROJECT 4: BINARY STUDENT DATABASE — FULL CRUD

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

struct Student { int id; char name[50]; float gpa; bool active; };
const string DB = "students.db";

// CREATE — add student
void addStudent(const Student& s) {
    ofstream f(DB, ios::binary|ios::app);
    f.write((char*)&s, sizeof(s));
}

// READ — find by id  (O(n) scan)
bool findStudent(int id, Student& out) {
    ifstream f(DB, ios::binary);
    Student s;
    while (f.read((char*)&s, sizeof(s)))
        if (s.id==id && s.active) { out=s; return true; }
    return false;
}

// UPDATE — update GPA (direct seek)
bool updateGPA(int id, float newGPA) {
    fstream f(DB, ios::binary|ios::in|ios::out);
    Student s; streampos p;
    while ((p=f.tellg()), f.read((char*)&s, sizeof(s)))
        if (s.id==id) { s.gpa=newGPA; f.seekp(p);
f.write((char*)&s,sizeof(s)); return true; }
    return false;
}

// DELETE — soft delete (mark active=false)
bool deleteStudent(int id) {
    fstream f(DB, ios::binary|ios::in|ios::out);
    Student s; streampos p;
    while ((p=f.tellg()), f.read((char*)&s, sizeof(s)))
        if (s.id==id) { s.active=false; f.seekp(p);
f.write((char*)&s,sizeof(s)); return true; }
    return false;
}

// LIST — all active students
void listStudents() {
    ifstream f(DB, ios::binary); Student s; int n=0;
    cout << "\nID   Name         GPA\n" << string(35,'-') << "\n";
    while (f.read((char*)&s, sizeof(s)))
      if (s.active) {printf("%-5d %-24s %.2f\n", s.id, s.name,s.gpa); n++;
}
    cout << n << " student(s) found.\n";
}

int main() {
    addStudent({1,"Ali Khan",3.8f,true});
    addStudent({2,"Sara Baig",3.9f,true});
    addStudent({3,"Zain Mir",3.5f,true});
    listStudents();
    updateGPA(2, 3.95f);   cout << "Updated Sara's GPA.\n";
    deleteStudent(3);       cout << "Deleted Zain.\n";
    listStudents();
}
```

PROJECT 3: Simple File Encryption (XOR Cipher)

PROJECT 5: XOR FILE ENCRYPTION

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// XOR encryption — symmetric (same key
encrypts & decrypts)
void xorFile(const string& inFile,
             const string& outFile,
             const string& key) {
    ifstream fin(inFile,  ios::binary);
    ofstream fout(outFile, ios::binary);

    if (!fin||!fout) {
        cerr<<"File open failed!\n"; return;
    }

    char ch;
    size_t ki = 0;
    while (fin.get(ch)) {
        // XOR each byte with cycling key
        char enc = ch ^ key[ki % key.size()];
        fout.put(enc);
        ki++;
    }
    cout << "Done: " << inFile
         << " -> " << outFile << "\n";
}

bool filesIdentical(const string& a, const
string& b){
    ifstream fa(a,ios::binary),
fb(b,ios::binary);
    return equal(
      istreambuf_iterator<char>(fa),
      istreambuf_iterator<char>(),
      istreambuf_iterator<char>(fb));
}

int main() {
    string key = "MySuperKey123";

    // Encrypt
    xorFile("secret.txt",
            "secret.enc", key);

    // Decrypt (XOR again = original)
    xorFile("secret.enc",
            "secret_dec.txt", key);

    cout << "Match: "
         << filesIdentical("secret.txt",
                           "secret_dec.txt")
         << "\n";  // 1 = identical!
}
```

HOW XOR WORKS

Original File

H e l l o

Key (cycled)

M y S u p

XOR ⊕

⊕ ⊕ ⊕ ⊕ ⊕

Encrypted

05 17 3f 19 3f

XOR again ⊕

⊕ ⊕ ⊕ ⊕ ⊕

Decrypted ✓

H e l l o

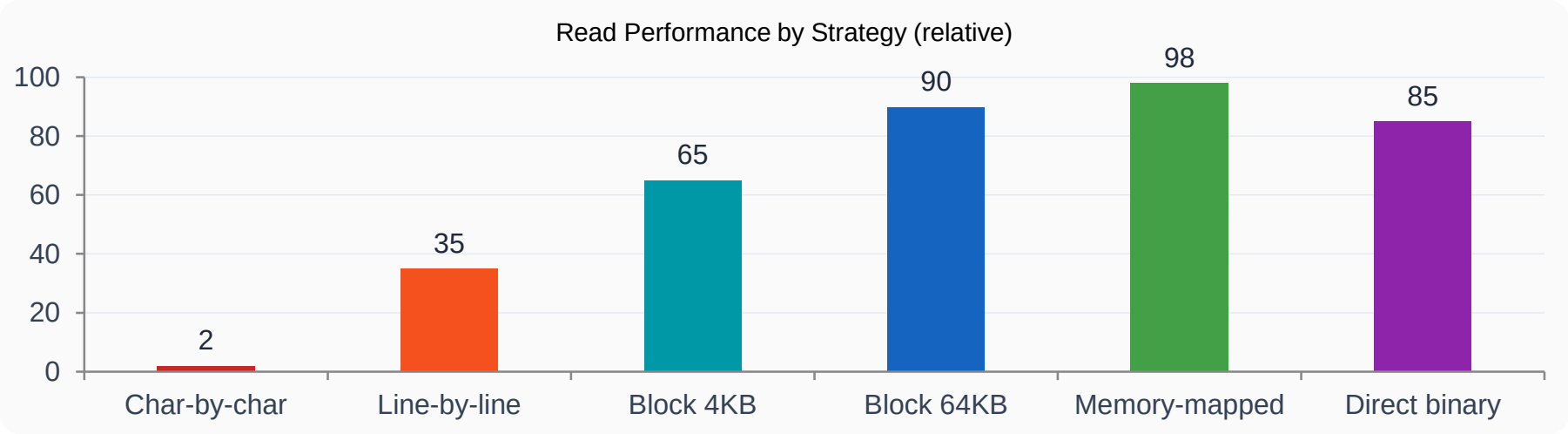
⚠ *For production use:*
AES-256 via OpenSSL or libsodium.
XOR is illustrative only.

SECTION 09

Best Practices & Performance

Tips • Pitfalls • Portability • Security • Summary

PERFORMANCE TIPS FOR FILE I/O



Use block reads
Read 4KB–64KB chunks with read() instead of character-by-character — 45× faster!

Flush strategically
flush() forces disk write but is slow. Use only when data persistence is critical.

Preallocate with seekp
For large files, seekp to end first to avoid repeated reallocation on many writes.

Binary over text
Binary avoids text parsing overhead. Write structs directly with write().

Reuse stream objects
Opening/closing files has overhead. Keep streams open for batch operations.

Use sync_with_stdio
sync_with_stdio(false) speeds cout but may affect mixing with printf — test first.

COMMON PITFALLS & HOW TO AVOID THEM

1. Not checking is_open()

✗ WRONG

```
// Not checking if file opened
ifstream fin("data.txt");
string s;
fin >> s; // crashes if file
missing!
```

✓ RIGHT

```
ifstream fin("data.txt");
if (!fin.is_open()) {
    cerr << "File not found!\n";
    return 1;
}
fin >> s; // safe now
```

2. Resource / Flush Leaks

✗ WRONG

```
// Resource leak – never closed
void processFile() {
    ofstream f("out.txt");
    f << "data";
    // f never closed – flush
delayed!
}
```

✓ RIGHT

```
void processFile() {
    ofstream f("out.txt"); //
RAII
    f << "data";
}   // destructor closes &
flushes
// Or call f.close() explicitly
```

3. Wrong Mode for Binary Data

✗ WRONG

```
// Text mode for binary data
ofstream f("img.png");    //
WRONG!
f.write(imgData, size);
// On Windows \n→\r\n corrupts
image
```

✓ RIGHT

```
// Binary mode for binary data
ofstream f("img.png",
           ios::binary); //
CORRECT
f.write(imgData, size);
// Exact bytes preserved
```

4. Misusing eof() Loop

✗ WRONG

```
// Infinite loop on eof()
while (!fin.eof()) { // WRONG!
    fin >> data;     // processes
    process(data);// last item
                  //twice!
}
```

✓ RIGHT

```
// Correct eof handling
while (fin >> data) { // CORRECT
    process(data);
}
// Loop exits when read fails
(eof/error)
```

SECURITY CONSIDERATIONS IN FILE HANDLING

Path Traversal Attack

NEVER use user input directly as a filename. Input like "../../etc/passwd" can access system files.
Fix: Validate and sanitize filenames, use basenames only.

Sensitive Data in Files

Never store plain-text passwords or keys in files.
Fix: Hash passwords (bcrypt), encrypt sensitive fields, use OS-level file permissions.

Buffer Overflow in Reading

Using char arrays with get/read without size limits causes overflows.
Fix: Always specify buffer size: fin.read(buf, sizeof(buf)-1); or use std::string.

File Permissions

Files with 777 permissions on Linux expose data to all users.
Fix: Create files with appropriate permissions (chmod 600 for sensitive data).

Race Conditions (TOCTOU)

Check-then-use race: file exists at check, deleted before open.
Fix: Open directly and check result; avoid separate exists() + open() pattern.

Secure File Deletion

os remove() doesn't zero-fill — data remains on disk for recovery.
Fix: Overwrite with zeros before deleting sensitive files.

⚠ Security Rule: Treat file I/O like network I/O — validate all inputs, check all return values, minimize permissions (principle of least privilege).

PRACTICE PROBLEMS

01

File Copy Utility

Copy any file byte-by-byte using binary mode. Print progress percentage during copy. Handle source not found.

02

Word Count Program

Mimic 'wc' command: count lines, words, and characters in a text file. Read any filename from command-line args.

03

Phone Book (Text)

Store name+number per line. Add, search, delete entries. Load on start, save on exit. Prevent duplicate numbers.

04

Binary Inventory System

Product struct with id, name, price, qty. Full CRUD with binary files. Sort by price. Export to CSV text file.

05

Multi-File Log Analyser

Read multiple .log files, parse timestamps, filter by date range, count errors per hour.

06

Mini Database Engine

CSV 'table' with create/insert/select/delete. Support WHERE conditions. Index file for O(1) lookup. Handle concurrent access.

COMPLETE QUICK REFERENCE CARD

Stream Classes

```
ifstream  – read files
ofstream  – write files
fstream   – read+write

#include <fstream>

Constructors:
ifstream fin("f.txt");
ofstream fout("f.txt");
fstream rw("f.txt",
  ios::in|ios::out);
```

Open Modes

```
ios::in     – read
ios::out    – write
ios::app    – append
ios::ate    – at end
ios::trunc  – truncate
ios::binary – binary

Combine with |:
ios::in|ios::binary
ios::out|ios::app
```

Key Methods

```
// State
is_open()  fail()  eof()
bad()  good()  clear()

// Positioning
seekg(pos)  seekg(off,dir)
seekp(pos)  tellg()  tellp()

// I/O
getline(fin,str)
fin.read(buf,n)
fout.write(buf,n)
fin.get(ch)
fout.put(ch)
fin.gcount()
```

KEY TAKEAWAYS & SUMMARY

3
Stream Classes

6
File Open Modes

4
Read Techniques

ESSENTIAL TAKEAWAYS

- Always verify file opened with is_open() before any I/O — never assume success
- Use RAII: stream destructor closes file automatically — prefer scope-based management
- Text mode for human-readable data; binary mode for structs, images, and raw bytes
- Never use while(!fin.eof()) — use while(fin >> data) or while(getline(fin, line))
- seekg/seekp enable O(1) random access to any record in a binary file — crucial for DBs
- ios::app guarantees appends are atomic — safe for log files; ios::out truncates by default
- For performance: block reads (read()) are 45× faster than character-by-character reads
- Sanitize all user-supplied filenames — path traversal (../../etc/passwd) is a real threat

THANK YOU

File Handling in C++

Streams • Modes • Text I/O • Binary I/O • Pointers • Error Handling • Real Projects • Best Practices